LAPORAN TUGAS PEKAN 5

STRUKTUR DATA

Disusun oleh:

FAHRI SYAH PUTRA RAMADHAN

2511531015

Dosen pengampu:

Dr. Wahyudi. MT

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG, 2026

PENJELASAN KODE PROGRAM

```java
package pekan5_2511531015;

public class HapusSLL_2511531015 {
//fungsi untuk menghapus head
    public static NodeSLL_2511531015 deleteHead(NodeSLL_2511531015 head) {
        // jika SLL kosong
        if (head == null)
            return null;
    //pindahkan head ke mode berikutnya
        head = head.next_1015;
        //return head baru
        return head;    }
    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511531015 removeLastNode(NodeSLL_2511531015 head) {
        // jika list kosong, return null
        if (head == null) {
            return null;

        }
        //jika list satu node, hapus node dan return null
        if (head.next_1015 == null) {
            return null;

        }
        //temukan node terakhir ke dua
        NodeSLL_2511531015 secondLast = head;
        while (secondLast.next_1015.next_1015 != null) {
            secondLast = secondLast.next_1015;

        }
        //hapus node terakhir
        secondLast.next_1015 = null;
        return head;
    }
    //fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511531015 deleteNode(NodeSLL_2511531015 head, int position) {
        NodeSLL_2511531015 temp_1015 = head;
        NodeSLL_2511531015 prev_1015 = null;
        //jika linked list null
        if (temp_1015 == null)
            return head;
        //kasus 1: head dihapus
        if(position == 1) {
            head = temp_1015.next_1015;
            return head;}
            //kasus 2: menghapus node di tengah
            //telusuri ke node yang dihapus
            for (int i = 1; temp_1015 != null && i <position; i++) {
```

```java
                return head;    }

                //fungsi mencetak SLL
            public static void printList(NodeSLL_2511531015 head) {
                NodeSLL_2511531015 curr = head;
                while(curr.next_1015 !=null) {
                    System.out.print(curr.data_1015 + "-->");
                curr = curr.next_1015;  }
                if(curr.next_1015==null) {
                System.out.print(curr.data_1015 ); }
                System.out.println(); }



//kelas main
public static void main (String[] args) {
    //buat SLL 1 -> 2-> 3-> 4-> 5 -> 6 -> null
    NodeSLL_2511531015 head = new NodeSLL_2511531015(1);
    head.next_1015 = new NodeSLL_2511531015(2);
    head.next_1015.next_1015 = new NodeSLL_2511531015(3);
    head.next_1015.next_1015.next_1015 = new NodeSLL_2511531015(4);
    head.next_1015.next_1015.next_1015.next_1015 = new NodeSLL_2511531015(5);
    head.next_1015.next_1015.next_1015.next_1015.next_1015 = new NodeSLL_2511531015(6);
    //cetak list awal
    System.out.println("list awal: ");
    printList(head);
    //hapus head
    head = deleteHead(head);
    System.out.println("list setelah head dihapus: ");
    printList(head);
    //hapus node terakhir
    head = removeLastNode(head);
    System.out.println("list setelah simpul terakhir di hapus: ");
    printList(head);
    //deleting node at pposition 2
    int position = 2;
    head = deleteNode(head, position);
    //print list after deletion
    System.out.println("List setelah posisi 2 dihapus; ");
    printList(head);


}

}
```

Kode program tersebut berfungsi untuk mengelola **Singly Linked List (SLL)**, dengan berbagai operasi penghapusan node seperti menghapus head, node terakhir, dan node pada posisi tertentu. Berikut adalah penjelasan singkat bagian-bagian penting dari program tersebut:

1. **deleteHead(NodeSLL head)**:
    - Fungsi ini menghapus node pertama (head) dari linked list.
    - Jika SLL kosong, mengembalikan null.
    - Mengubah head ke node berikutnya, sehingga head yang lama terhapus.

2. **removeLastNode(NodeSLL head)**:
    - Fungsi ini menghapus node terakhir dari linked list.
    - Jika list kosong atau hanya memiliki satu node, mengembalikan null.
    - Jika lebih dari satu node, mencari node kedua terakhir dan menghapus referensi ke node terakhir.
3. **deleteNode(NodeSLL head, int position)**:
    - Fungsi ini menghapus node di posisi tertentu dalam linked list.
    - Jika posisi adalah 1 (head), maka head dihapus.
    - Jika posisi lebih dari 1, traversing dilakukan hingga node yang sesuai ditemukan, kemudian menghapus node tersebut.
4. **printList(NodeSLL head)**:
    - Fungsi ini mencetak isi linked list mulai dari head hingga node terakhir.
5. **main(String[] args)**:
    - Di sini dibuat contoh linked list dengan 6 node yang berisi data dari 1 hingga 6.
    - Menampilkan list awal, menghapus head, menghapus node terakhir, dan menghapus node pada posisi ke-2, lalu mencetak list setelah setiap penghapusan.

Output nya:

```
list awal:
1-->2-->3-->4-->5-->6
list setelah head dihapus:
2-->3-->4-->5-->6
list setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus;
2-->4-->5
```

```java
package pekan5_2511531015;

public class NodeSLL_2511531015 {
    //node bagian data
    int data_1015;
    //pointer ke node selanjutnya
    NodeSLL_2511531015 next_1015;
    //konstruktor menginisialisasi node dengan data
    public NodeSLL_2511531015(int data_1015)
    {
        this.data_1015 = data_1015;
        this.next_1015 = null;


    }
    {

    }

}

```

Program di atas mendefinisikan **Node** untuk **Singly Linked List (SLL)**, yang berfungsi untuk menyimpan data dan referensi ke node berikutnya dalam linked list. Berikut adalah penjelasan bagian-bagian penting dari kode tersebut:

1. **Variabel Instance**:
    - **int data_1015**: Variabel ini menyimpan data yang disimpan dalam node. Dalam contoh ini, tipe data yang disimpan adalah integer.
    - **NodeSLL_2511531015 next_1015**: Variabel ini adalah pointer atau referensi ke node berikutnya dalam linked list. Jika node ini adalah node terakhir, maka next_1015 bernilai null.
2. **Konstruktor**:
    - **public NodeSLL_2511531015(int data_1015)**: Konstruktor ini digunakan untuk membuat node baru dengan nilai data yang diberikan (parameter data_1015), dan menginisialisasi next_1015 ke null (karena saat pembuatan node baru, node tersebut belum terhubung ke node lain).
3. **Inisialisasi Default**:
    - Blok {} kosong setelah konstruktor digunakan untuk deklarasi dan inisialisasi lebih lanjut jika diperlukan. Namun, pada kode ini blok tersebut tidak digunakan untuk sesuatu yang spesifik.

```java
package pekan5_2511531015;

public class PencarianSLL_2511531015 {
    static boolean searchKey_1015(NodeSLL_2511531015 head, int key_1015) {
        NodeSLL_2511531015 curr = head;
        while (curr != null) {
            if(curr.data_1015 == key_1015)
                return true;
            curr = curr.next_1015; }
        return false;   }
    public static void traversal(NodeSLL_2511531015 head) {
        //mulai dari head
        NodeSLL_2511531015 curr = head;
        //telusuri sampai pointer null
        while (curr != null) {
            System.out.print(" "+ curr.data_1015);
            curr = curr.next_1015;  }
        System.out.println();   }
        public static void main (String[] args) {
            NodeSLL_2511531015 head = new NodeSLL_2511531015(14);
            head.next_1015 = new NodeSLL_2511531015(21);
            head.next_1015.next_1015 = new NodeSLL_2511531015(13);
            head.next_1015.next_1015.next_1015 = new NodeSLL_2511531015(30);
            head.next_1015.next_1015.next_1015.next_1015 = new NodeSLL_2511531015(10);
            System.out.print("penulusuran SLL : ");
            traversal(head);
            //data yang akan dicari
            int key_1015 = 30;
            System.out.print("cari data "+key_1015+ " = ");
            if (searchKey_1015(head, key_1015))
                System.out.println("ketemu");
            else
                System.out.println("Tidak ada");
        }
    }
```

1. searchKey_1015(NodeSLL head, int key_1015):

- Fungsi ini digunakan untuk mencari apakah sebuah nilai (key_1015) ada dalam **Singly Linked List (SLL)**.
- **Parameter**:
    - head: Merupakan node pertama dari linked list.
    - key_1015: Nilai yang akan dicari dalam linked list.
- **Proses**:
    - Fungsi ini menggunakan perulangan while untuk menelusuri seluruh node dalam list. Dimulai dari node head, lalu mengecek apakah data_1015 di setiap node sama dengan key_1015.
    - Jika ditemukan, fungsi akan mengembalikan true. Jika tidak, perulangan berlanjut hingga mencapai node terakhir (yaitu null).
    - Jika tidak ditemukan, fungsi mengembalikan false.

2. traversal(NodeSLL head):

- Fungsi ini digunakan untuk menelusuri dan mencetak semua nilai yang ada dalam linked list.
- **Parameter**:
    - head: Merupakan node pertama dari linked list.

- **Proses**:
    - Fungsi ini melakukan traversal dari node head hingga node terakhir dengan menggunakan perulangan while. Pada setiap iterasi, data node (yaitu data_1015) dicetak.
    - Jika list kosong (head == null), maka perulangan tidak akan terjadi, dan fungsi langsung selesai.

3. main(String[] args):

- Fungsi main digunakan untuk mengeksekusi program dan memberikan contoh penggunaan dari fungsi-fungsi yang sudah dibuat.
- **Proses**:
    - Linked list head dibangun dengan lima node yang memiliki data: 14, 21, 13, 30, dan 10.
    - Fungsi **traversal** dipanggil untuk mencetak isi linked list yang sudah dibangun.
    - Program kemudian mencari data 30 menggunakan fungsi **searchKey_1015**.
    - Jika data ditemukan, program akan mencetak "ketemu". Jika tidak, akan mencetak "Tidak ada".

```java
package pekan5_2511531015;

public class TambahSLL_2511531015 {
public static NodeSLL_2511531015 insertAtFront(NodeSLL_2511531015 head, int value) {
    NodeSLL_2511531015 new_node = new NodeSLL_2511531015(value);
    new_node.next_1015 = head;
    return new_node;

}
//fungsi menambahkan node di akhir SLL
public static NodeSLL_2511531015 insertAtEnd(NodeSLL_2511531015 head, int value) {
    //buat sebuah node dengan sebuah nilai
    NodeSLL_2511531015 newNode = new NodeSLL_2511531015(value);
    //jika list kosong maka node jadi head
    if (head == null) {
        return newNode;

    }
    // simpan head ke variabel sementara
    NodeSLL_2511531015 last = head;
    //telusuri ke node akhir
    while (last.next_1015 != null) {
        last = last.next_1015;

    }
    //ubah pointer
    last.next_1015 = newNode;
    return head;

}
static NodeSLL_2511531015 GetNode(int data) {
    return new NodeSLL_2511531015(data);

}
static NodeSLL_2511531015 insertPos(NodeSLL_2511531015 headNode, int position, int value) {
    NodeSLL_2511531015 head = headNode;
    if(position < 1)
        System.out.print("invalid position");
    if(position == 1) {
        NodeSLL_2511531015 new_node = new NodeSLL_2511531015(value);
    new_node.next_1015 = head;
    return new_node;
} else {
    while (position-- != 0) {
        if(position == 1) {
            NodeSLL_2511531015 newNode = GetNode(value);
            newNode.next_1015 = headNode.next_1015;
            headNode.next_1015 = newNode;
```

```java
    if(position != 1)
        System.out.print("posisi di luar jangkauan");   }
    return head; }
    public static void printList(NodeSLL_2511531015 head) {
        NodeSLL_2511531015 curr = head;
        while (curr.next_1015 != null) {
            System.out.print(curr.data_1015+"-->");
            curr = curr.next_1015;

        }
        if(curr.next_1015==null) {
            System.out.print(curr.data_1015); }
        System.out.println();


    }
    public static void main(String[] args) {
        //buat linked List 2->3->5->6
        NodeSLL_2511531015 head =new NodeSLL_2511531015(2);
        head.next_1015 = new NodeSLL_2511531015(3);
        head.next_1015.next_1015 = new NodeSLL_2511531015(5);
        head.next_1015.next_1015.next_1015 = new NodeSLL_2511531015(6);
        //cetak List Asli
        System.out.print("Senarai berantai awal");
        printList(head);
        //tambahkan node baru di depan
        System.out.print("tambah 1 simpul di depan: ");
        int data_1015 = 1;
        head = insertAtFront(head, data_1015);
        // cetak update list
        printList(head);
        //tambahkan node baru di belakang
        System.out.print("tambah 1 simpul di belakang: ");
        int data2_1015 = 7;
        head = insertAtEnd(head, data2_1015);
        //cetak update list
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_1015 = 4;
        int pos=4;
        head = insertPos(head,pos,data3_1015);
        //cetak update list
        printList(head);


    }

}
```

```java
package pekan5_2511531015;

public class TambahSLL_2511531015 {
public static NodeSLL_2511531015 insertAtFront(NodeSLL_2511531015 head, int value) {
    NodeSLL_2511531015 new_node = new NodeSLL_2511531015(value);
    new_node.next_1015 = head;
    return new_node;
}
//fungsi menambahkan node di akhir SLL
public static NodeSLL_2511531015 insertAtEnd(NodeSLL_2511531015 head, int value) {
    //buat sebuah node dengan sebuah nilai
    NodeSLL_2511531015 newNode = new NodeSLL_2511531015(value);
    //jika list kosong maka node jadi head
    if (head == null) {
        return newNode;

    }
    // simpan head ke variabel sementara
    NodeSLL_2511531015 last = head;
    //telusuri ke node akhir
    while (last.next_1015 != null) {
        last = last.next_1015;

    }
    //ubah pointer
    last.next_1015 = newNode;
    return head;

}
static NodeSLL_2511531015 GetNode(int data) {
    return new NodeSLL_2511531015(data);

}
static NodeSLL_2511531015 insertPos(NodeSLL_2511531015 headNode, int position, int value) {
    NodeSLL_2511531015 head = headNode;
    if(position < 1)
        System.out.print("invalid position");
    if(position == 1) {
        NodeSLL_2511531015 new_node = new NodeSLL_2511531015(value);
    new_node.next_1015 = head;
    return new_node;
} else {
    while (position-- != 0) {
        if(position == 1) {
            NodeSLL_2511531015 newNode = GetNode(value);
            newNode.next_1015 = headNode.next_1015;
            headNode.next_1015 = newNode;
            break;
```

```java
    if(position != 1)
        System.out.print("posisi di luar jangkauan");   }
    return head; }
    public static void printList(NodeSLL_2511531015 head) {
        NodeSLL_2511531015 curr = head;
        while (curr.next_1015 != null) {
            System.out.print(curr.data_1015+"-->");
            curr = curr.next_1015;

        }
        if(curr.next_1015==null) {
            System.out.print(curr.data_1015); }
        System.out.println();


    }
    public static void main(String[] args) {
        //buat linked List 2->3->5->6
        NodeSLL_2511531015 head =new NodeSLL_2511531015(2);
        head.next_1015 = new NodeSLL_2511531015(3);
        head.next_1015.next_1015 = new NodeSLL_2511531015(5);
        head.next_1015.next_1015.next_1015 = new NodeSLL_2511531015(6);
        //cetak List Asli
        System.out.print("Senarai berantai awal");
        printList(head);
        //tambahkan node baru di depan
        System.out.print("tambah 1 simpul di depan: ");
        int data_1015 = 1;
        head = insertAtFront(head, data_1015);
        // cetak update list
        printList(head);
        //tambahkan node baru di belakang
        System.out.print("tambah 1 simpul di belakang: ");
        int data2_1015 = 7;
        head = insertAtEnd(head, data2_1015);
        //cetak update list
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_1015 = 4;
        int pos=4;
        head = insertPos(head,pos,data3_1015);
        //cetak update list
        printList(head);


    }

}
```

1. insertAtFront(NodeSLL head, int value):

- Fungsi ini digunakan untuk menambahkan node baru di depan (head) dari **Singly Linked List (SLL)**.
- **Proses**:
    - Membuat node baru dengan nilai value.
    - Node baru ini diset sebagai head, dan referensi next_1015 node baru ini diarahkan ke node sebelumnya (head lama).
    - Mengembalikan new_node sebagai head yang baru.

2. insertAtEnd(NodeSLL head, int value):

- Fungsi ini digunakan untuk menambahkan node baru di akhir linked list.
- **Proses**:
    - Membuat node baru dengan nilai value.
    - Jika list kosong (head == null), node baru langsung menjadi head.
    - Jika tidak, fungsi menelusuri list hingga mencapai node terakhir dan menghubungkannya dengan node baru.
    - Mengembalikan head, yang tetap menjadi node pertama.

3. GetNode(int data):

- Fungsi ini hanya membuat dan mengembalikan sebuah node baru dengan nilai data yang diberikan.
- Fungsi ini digunakan untuk mempermudah pembuatan node dalam fungsi lain.

4. insertPos(NodeSLL headNode, int position, int value):

- Fungsi ini menambahkan node baru pada posisi tertentu dalam linked list.
- **Proses**:
    - Fungsi pertama-tama memeriksa apakah posisi valid.
    - Jika posisi adalah 1 (di depan), node baru ditambahkan di depan.
    - Jika posisi lebih dari 1, fungsi menelusuri list hingga menemukan posisi yang tepat dan menyisipkan node baru di posisi tersebut.
    - Jika posisi di luar jangkauan, fungsi akan mencetak pesan error.

5. printList(NodeSLL head):

- Fungsi ini mencetak seluruh isi linked list mulai dari head hingga node terakhir.
- Fungsi ini menelusuri setiap node dan mencetak nilai data_1015 dari setiap node, diikuti dengan panah -->.

6. main(String[] args):

- Fungsi main digunakan untuk mendemonstrasikan penggunaan fungsi-fungsi di atas.
- **Proses**:
  - Membuat linked list awal dengan nilai: 2 → 3 → 5 → 6.
  - Menambahkan node baru dengan nilai 1 di depan list.
  - Menambahkan node baru dengan nilai 7 di akhir list.
  - Menambahkan node baru dengan nilai 4 di posisi ke-4.
  - Setelah setiap perubahan, fungsi **printList** dipanggil untuk menampilkan hasil update list.

  Dengan Output:

```
Senarai berantai awal2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6 p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.exe (May 7, 2026, 3:12:07 PM – 3:12:09 PM elapsed: 0:00:02.072) [pid
tambah 1 simpul di belakang: tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```